```
➤   expenses = JSON.parse(last_response.body)
➤   expect(expenses).to contain_exactly(coffee, zoo)
end
```

We're using the same techniques from before: driving the app, grabbing the last_response from Rack::Test, and looking at the results. There are lots of ways to compare collections in RSpec. Here, we want to check that the array contains the two expenses we want—and only those two—without regard to the order. The contain_exactly matcher captures this requirement.

If we had a specific business requirement that the expenses be in a certain sequence, we could compare the collections with eq instead, as in eq [coffee, zoo]. Here, we don't care about the order. Using a matcher more flexible than eq makes our spec more resilient, giving us the latitude to change the ordering in the future without fighting a broken test.

Go ahead and run the latest version of your spec:

```
$ bundle exec rspec
F

Failures:

  1) Expense Tracker API records submitted expenses
     Failure/Error: expect(last_response.status).to eq(200)

       expected: 200
            got: 404

       (compared using ==)

« truncated »
```

Since we haven't defined a way for clients to read back data yet, we're getting another 404 status code. Let's add a route to the Sinatra app that returns an empty JSON array:

04-acceptance-specs/05/expense_tracker/app/api.rb

```
get '/expenses/:date' do
  JSON.generate([])
end
```

Now, when we rerun our specs, we get an incorrect response, rather than an HTTP error:

```
$ bundle exec rspec
F

Failures:

  1) Expense Tracker API records submitted expenses
     Failure/Error: expect(expenses).to contain_exactly(coffee, zoo)
```